



Universidad: Universidad Nacional Del Nordeste

Trabajo Práctico Integrador 2026

Sistema De Semáforo Inteligente y Análisis Comparativo de Paradigmas

Grupo N° 28

Integrantes:

- Meza Franco Leonel
- Ojeda Cynthia Magali
- Rivero Alessandro Sebastián
- Torres Martino Elías Joaquín

Fecha De Entrega: 16 de junio

Introducción

El objetivo de este trabajo es modelar un problema del mundo real que consiste en desarrollar un sistema de tráfico urbano en código LISP utilizando el paradigma funcional. Además, busca desarrollar una mayor autonomía y pensamiento algorítmico para resolver la integración de herramientas externas y el aprendizaje de nuevas tecnologías con la mínima asistencia docente, así como fomentar el trabajo en grupo.

También pretende mejorar el análisis crítico y la capacidad de evaluar las funciones creadas, comprendiendo sus implicancias en la memoria y el flujo de datos, y comparar cómo diferentes lenguajes abordan un mismo problema lógico.

Un Paradigma Funcional es un Modelo matemático de composición funcional donde el resultado de un cálculo es la entrada del siguiente, y así sucesivamente hasta que una composición produce el valor deseado. Como su nombre lo dice operan solamente a través de funciones. Cada función devuelve un solo valor, dada una lista de parámetros. La programación funcional proporciona la capacidad para que un programa se modifique así mismo, es decir que pueda aprender.

Para este trabajo se utilizó LISP porque fue el lenguaje solicitado para el desarrollo del trabajo ya que es un lenguaje que permite trabajar con el paradigma funcional de manera eficiente. Gracias a esto fue posible desarrollar el sistema de tráfico urbano, y así poder utilizar funciones para controlar los cambios de estado sin modificar los datos y transiciones del sistema.

Desarrollo del sistema

Modelado del problema:

Para realizar el modelado analizamos el funcionamiento de un semáforo real. Identificamos que está compuesto por tres estados principales: rojo, amarillo y verde, los cuales deben seguir una secuencia específica para garantizar un funcionamiento correcto. Por este motivo, desarrollamos una función que permite al operador realizar cambios de estado, pero únicamente respetando las transiciones válidas del sistema: de rojo a amarillo, de amarillo a verde y de verde a rojo. Además, siguiendo las consignas del trabajo, el comportamiento del semáforo debía ser cíclico, por lo que una vez completada la secuencia vuelve a comenzar nuevamente desde el estado rojo.

A partir de esta idea desarrollamos una función que permite identificar en qué estado se encuentra el semáforo según el tiempo transcurrido. Como cada luz debe permanecer activa durante una cantidad específica de segundos, definimos una duración para cada fase y utilizamos esos valores para determinar el color correspondiente en cada momento.

Posteriormente incorporamos una función para registrar los cambios de estado del semáforo. Esta función informa cuándo ocurre una transición y entre qué colores se produce, permitiendo llevar un seguimiento del funcionamiento del sistema.

También implementamos una función para calcular la duración total de un ciclo completo, sumando los tiempos asignados a las luces roja, amarilla y verde. Con esta información desarrollamos otra función encargada de evaluar si la duración del ciclo es adecuada. Para ello tomamos como referencia una duración recomendada y establecimos distintos criterios: si el ciclo es demasiado corto se considera no recomendado, si es demasiado largo también se

considera no recomendado y, si se encuentra dentro del rango óptimo, se clasifica como recomendado.

Además, calculamos la cantidad de ciclos que pueden realizarse en un período determinado de tiempo. Para ello se obtiene la duración total de un ciclo y se determina cuántas veces puede repetirse dentro del tiempo indicado.

Por último, elaboramos un informe de distribución que muestra el porcentaje de tiempo que corresponde a cada estado del semáforo. Esto permite analizar cómo se reparte el tiempo total entre las luces roja, amarilla y verde, y observar cuál de ellas permanece activa durante un mayor porcentaje del ciclo.

Estructura utilizada

Para el desarrollo del sistema se utilizaron diferentes estructuras propias de Common Lisp. En primer lugar, los estados del semáforo fueron representados mediante caracteres, como **en-rojo**, **en-amarillo** y **en-verde**, ya que permiten identificar de forma clara cada estado y facilitan las comparaciones dentro de las funciones.

También se utilizaron listas para agrupar y devolver información relacionada con las transiciones del semáforo. Estas listas permiten almacenar varios datos en una única estructura y son ampliamente utilizadas en la programación funcional.

Además, se emplearon listas numéricas para representar los tiempos de duración de cada fase del semáforo, las cuales son procesadas mediante funciones de orden superior para realizar cálculos como la duración total del ciclo y la distribución porcentual de cada color.

Por último, se utilizaron variables locales dentro de las funciones para realizar cálculos temporales sin modificar datos externos, respetando así los principios de inmutabilidad y evitando efectos secundarios en el programa.

Funciones implementadas

Para desarrollar el sistema de semáforos inteligentes se implementaron diversas funciones, cada una encargada de una tarea específica dentro de la lógica del programa.

La función **transición** se encarga de determinar el cambio de estado del semáforo, estableciendo cuál será la siguiente acción según el color actual y el color al que debe cambiar.

La función **timer** determina el color que debe tener el semáforo en función del tiempo transcurrido dentro de un ciclo completo, permitiendo automatizar el cambio entre los distintos estados.

La función **registrar-cambio** tiene la finalidad de registrar e informar los cambios de estado que ocurren durante la ejecución del sistema, generando mensajes descriptivos para la auditoría.

La función **duración-ciclo** calcula la duración total de un ciclo del semáforo sumando los tiempos correspondientes a cada fase.

La función **recomendación-ciclo** evalúa si la duración total del ciclo se encuentra dentro de los rangos recomendados, proporcionando una observación sobre la eficiencia del sistema.

La función **ciclos-por-tiempo** permite calcular cuántos ciclos completos puede realizar el semáforo en un período de tiempo determinado.

Por último, la función **informe-distribución** calcula el porcentaje de tiempo que representa cada color dentro del ciclo total, permitiendo analizar la distribución temporal de las distintas fases del semáforo.

Clasificación funcional

Función	Naturaleza	Estrategia	Impacto
Transcripción	Pura	Condicional	No Destructivo
Temporizador	Pura	Condicional	No Destructivo
Registrar cambio	Impura	Directa/ Ejecución Secuencial	No Destructiva
Duración ciclo	Pura	Recorrido recursivo a través de la lista	No Destructiva
Recomendación ciclo	pura	Función predicado/ condicional	No Destructiva
Ciclo por tiempo	Pura	Calculo Directo	No Destructiva
Informe Distribución	Pura	Orden superior	No Destructiva

Presentación del lenguaje SCALA

Scala es un lenguaje de programación moderno cuyo nombre proviene de scalable lenguaje. Fue diseñado para combinar dos paradigmas de programación: la programación orientada a objeto y la programación funcional. Gracias a esta combinación, permite desarrollar aplicaciones de manera flexible, aprovechando las ventajas de ambos enfoques.

Características principales:

Presenta diversas características que lo convierten en una opción atractiva para el desarrollo de software como por ejemplo: combina programación orientada a objetos y programación funcional, permite escribir código más corto y expresivo que otros lenguajes tradicionales, es un lenguaje escalable, adecuado tanto para proyectos pequeños como para aplicaciones empresariales complejos, posee un sistema de tipos estático que ayuda a detectar errores antes de ejecutar el programa.

Áreas de utilización:

Se utiliza principalmente en el desarrollo de aplicaciones backend, APIs REST, microservicios y aplicaciones web empresariales. También es empleado en el procesamiento de grandes volúmenes de datos (Big Data), ingeniería de datos, sistemas distribuidos, plataformas de streaming y servicios en la nube.

Además, tiene una fuerte presencia en el sector financiero y tecnológico, donde se utiliza para desarrollar sistemas bancarios, plataformas Fintech y aplicaciones de trading algorítmico.

Empresas que utilizan SCALA

scala ha sido utilizado por importantes empresas tecnológicas para desarrollar sistemas de gran escala y alto rendimiento. Entre ellas se encuentran X (Twitter), LinkedIn, Netflix, Airbnb, Spotify, The Guardian y Coursera, que han empleado este lenguaje en distintos servicios backend, plataformas distribuidas y sistemas de procesamiento de datos.

Comparación con Lisp:

Tanto Lisp como scala permiten aplicar conceptos de programación funcional, pero presentan diferencias importantes en su sintaxis, sistema de tipos y forma de desarrollar aplicaciones.

Lisp es un lenguaje de tipado dinámico es decir, que los tipos de datos se verifican durante la ejecución del programa. En cambio, scala utiliza tipado estático, permitiendo detectar muchos errores durante la compilación antes de ejecutar la aplicación.

En cuanto a la sintaxis, LISP utiliza una notación basada en paréntesis, característica de los lenguajes de la familia LISP. Por ejemplo: para realizar una suma se escribe (+ 2 3). Sin embargo scala utiliza una sintaxis similar a otros lenguajes como java o c#, que suele resultar mas familiar para muchos programadores.

Respecto con la programación funcional, ambos lenguajes soportan funciones de orden superior, como recursividad y composición de funciones. Sin embargo scala incorpora estas características de forma más integrada junto con herramientas de programación orientadas a objetos.

Otra diferencia importante es la inmutabilidad. En LISP es posible utilizar estructuras mutables o inmutables según las necesidades del desarrollador. En scala, aunque también existe la mutabilidad, se fomenta el uso de estructuras inmutables como practica recomendada dentro del paradigma funcional.

Y así ambos lenguajes permiten resolver problemas mediante programación funcional, pero scala ofrece mayor integración con tecnologías modernas y sistemas empresariales, mientras que Lisp destaca por su flexibilidad, simplicidad conceptual y capacidad para trabajar con estructuras simbólicas.

Implementación de la función transición

```
;;scala Trancision
|   ;; colorActual, cambiarA, list son los argumentos que se utiliza con la forma del argumento
def transicion(colorActual: String, cambiarA: String): List[String] = {
|   ;; ya que en scala no hay cond utilizamos if
|   if (colorActual == "en-rojo" && cambiarA == "amarillo") {
|       List(colorActual, "cambiar-a-amarillo")
|   } else if (colorActual == "en-amarillo" && cambiarA == "verde") {
|       List(colorActual, "cambiar-a-verde")
|   } else if (colorActual == "en-verde" && cambiarA == "rojo") {
|       List(colorActual, "cambiar-a-rojo")
|   } else {
|       List(colorActual, "accion-por-defecto")
|   }
| }
def main(args: Array[String]): Unit = {
|   println(transicion("en-rojo", "amarillo"))
| }
}
```

Explicación: Este código está escrito en **Scala** y simula una transición de estados para un semáforo o sistema de colores. La función principal, llamada transicion, recibe dos parámetros

de tipo String: colorActual, que representa el estado actual, y cambiarA, que indica el color al que se desea cambiar.

Dentro de la función se utiliza una estructura de decisiones if, else if y else para evaluar diferentes combinaciones de colores. Dependiendo de los valores recibidos, la función devuelve una lista (List[String]) que contiene el color actual y la acción que debe realizarse.

Implementación de la función Timer

```
;REQUERIMIENTO 2
;Scala transición
def timer(epoch: Int): String = { ; Define la funcion
    val posicion = epoch % 216 ;Crea la varariable "posicion" que contiene el valor de epoch dividido 216

    if (posicion < 90) "rojo"
    else if (posicion < 96) "amarillo"
    else "verde"
}
```

Explicación: Este código está reescrito en Scala.

Lo que hace este código es que recibe una variable epoch, y se especifica que es un entero. Dentro de la función se crea la variable “posición” que contiene el resultado de epoch al ser dividido por 216. Después compara esta variable “posición”, si el número es menor a 90 el color de semáforo es rojo. si estaba entre 91 y 96 el color al amarillo, y cualquier otro resultado retorna verde

Código con error 1:

```
;REQUERIMIENTO 2
(defun timer (epoch)
  (let ((posicion (mod epoch 216)))
    (cond
      ((< posicion 90) 'rojo)
      ((< posicion 96) 'amarillo)
      (t 'verde))))

; MÁS ABAJO (Requerimiento 7 / Iteración 2)
(defun timer (tiempo-unix)
  (let* ((dur-rojo (duracion-color 'en-rojo))
        ...))
```

causa del error: Common lisp permite redefinir funciones, pero la segunda definición sobrescribe a la primera, causando confusión y posible comportamiento inesperado.

Solución: Eliminar la primera definición del Requerimiento 2, o renombrarla como timer-base:

```
;REQUERIMIENTO 2 (renombrada)
(defun timer-base (epoch)
  (let ((posicion (mod epoch 216)))
    (cond
      ((< posicion 90) 'rojo)
      ((< posicion 96) 'amarillo)
      (t 'verde))))

; Iteración 2 (se mantiene como timer)
(defun timer (tiempo-unix)
  ...)
```

Código con error 2: Los parámetros se llaman color-actual y cambiar-a, pero en el format se usan color-anterior y color-nuevo (variables inexistentes).

```
(defun registrar-cambio (epoch color-actual cambiar-a)
  (format t "Tiempo ~A: la luz ha cambiado de ~A a ~A~%"
    epoch color-anterior color-nuevo))
```

Solución:

```
(defun registrar-cambio (epoch color-anterior color-nuevo)
  (format t "Tiempo ~A: la luz ha cambiado de ~A a ~A~%"
    epoch color-anterior color-nuevo))
```

Código con error 3: La función no fue actualizada para la Iteración 2 que introduce amarillo-intermitente.

```
(defun transicion (color-actual cambiar-a)
  (cond
    ((and (equal color-actual 'en-rojo)
          (equal cambiar-a 'amarillo))
     (list color-actual "cambiar-a-amarillo"))
    ((and (equal color-actual 'en-amarillo)
          (equal cambiar-a 'verde))
     (list color-actual "cambiar-a-verde"))
    ((and (equal color-actual 'en-verde)
          (equal cambiar-a 'rojo))
     (list color-actual "cambiar-a-rojo"))
    (t (list color-actual 'accion-por-defecto))))
```

Solución:

```
(defun transicion (color-actual cambiar-a)
  (cond
    ;; Transiciones con intermitencia
    ((and (equal color-actual 'en-rojo)
          (equal cambiar-a 'amarillo-intermitente))
     (list color-actual "cambiar-a-amarillo-intermitente"))
    ((and (equal color-actual 'amarillo-intermitente)
          (equal cambiar-a 'verde))
     (list color-actual "cambiar-a-verde"))
    ((and (equal color-actual 'en-verde)
          (equal cambiar-a 'amarillo-intermitente))
     (list color-actual "cambiar-a-amarillo-intermitente"))
    ((and (equal color-actual 'amarillo-intermitente)
          (equal cambiar-a 'amarillo))
     (list color-actual "cambiar-a-amarillo"))
    ((and (equal color-actual 'en-amarillo)
          (equal cambiar-a 'amarillo-intermitente))
     (list color-actual "cambiar-a-amarillo-intermitente"))
    ((and (equal color-actual 'amarillo-intermitente)
          (equal cambiar-a 'rojo))
     (list color-actual "cambiar-a-rojo"))
    (t
     (list color-actual 'accion-por-defecto))))
```

Código con error 4: La función suma (first tiempos-fase) con el resultado de llamarse a sí misma

- Common Lisp debe recordar sumar cada valor después de que termine la recursión
- Esto crea una cadena de llamadas pendientes en la pila de ejecución
- Con listas muy largas (ej: 100,000 elementos), la pila se llena y explota

```
(defun duracion-ciclo (tiempos-fase)
  (cond
    ((endp tiempos-fase) 0)
    ((numberp (first tiempos-fase))
     (+ (first tiempos-fase) (duracion-ciclo (cdr tiempos-fase))))
    ) ; ← PROBLEMA AQUÍ
  (t (duracion-ciclo (cdr tiempos-fase))))
```

Solución: Usar reduce.

```
(defun duracion-ciclo (tiempos-fase)
  "Versión funcional con reduce."
  (reduce #'+ (remove-if-not #'numberp tiempos-fase)))
```

Comparación de scala y Lisp

Comparar Lisp y Scala para manipular listas es, básicamente, chocar con dos filosofías distintas. Siendo sincero, Scala te hace la vida mucho más fácil para procesar datos, y acá te digo por qué:

Lo que más pesa es cómo se lee el código. En Lisp tenés que leer de adentro hacia afuera; si encadenás varias funciones, tenés que irte hasta el centro de los paréntesis para entender qué rayos se ejecutó primero. Se vuelve un lío de paréntesis que te termina quemando la cabeza. En Scala, en cambio, la lógica es lineal: lista -> filtra -> mapea. Lo lees de izquierda a derecha, como una tubería, y el cerebro lo procesa al toque porque sigue el orden de los pasos que tenés en la cabeza.

Después está el tema de cómo tratás los datos. En Scala, al usar `.filter` o `.map` sobre la lista, sentís que el método es parte del objeto, es súper intuitivo. En Lisp, tenés que pasar la lista como argumento a una función externa como `mapcar`. Es más "matemático" o "puro" si querés, pero te obliga a hacer más gimnasia mental para mantener el hilo de lo que estás haciendo cuando el código crece un poco.

Y no olvidemos la sintaxis. Scala tiene esos atajos con los guiones bajos (`_`) que hacen que las funciones pequeñas sean una pavada de escribir. En Lisp, aunque podés usar `lambda` o referencias a funciones, terminás escribiendo mucho más y se vuelve verboso al divino botón.

En definitiva, Lisp tiene una elegancia conceptual tremenda, pero Scala gana por goleada en cuanto a legibilidad y agilidad para el día a día.

Conclusiones:

Con este trabajo logramos desarrollar un sistema de tráfico urbano utilizando Lisp y aplicando los principios de la programación funcional. Implementamos distintas funciones que permiten controlar los estados del semáforo, calcular la duración de los ciclos, registrar los cambios y analizar la distribución del tiempo de funcionamiento.

A lo largo del proyecto, pudimos poner en práctica conceptos como las funciones puras, la inmutabilidad, el manejo de listas y la organización modular del código. Además, fortalecimos nuestra capacidad de análisis y resolución de problemas al trabajar con un lenguaje diferente a los que utilizamos habitualmente.

El estudio comparativo con Scala también nos permitió identificar las diferencias y similitudes entre ambos lenguajes, comprendiendo cómo cada uno aborda la programación funcional y en qué contextos puede resultar más conveniente su utilización.

Bibliografía utilizada

Scala language Documentation

Scala official website

Programming in scala